

INSTITUTE OF MATHEMATICS AND COMPUTER SCIENCE

University of Sindh, Jamshoro

BS (CS) Part-I / MSc (Prev.) — 1st Semester Practical Examination 2023

Programming Fundamentals CC-311 — Complete Solutions

Instructions: Attempt any FOUR questions. All questions carry equal marks. Time: 60 Minutes.

Q1. Decreasing Alphabet Triangle

Problem: Write a C program to produce the following output:

```
A B C D E F G
A B C D E F
A B C D E
A B C D
A B C
A B
A B
A
```

Concept: Nested loops — outer loop controls rows, inner loop prints letters.

```
#include <stdio.h>

int main() {
    int i, j;
    /* Outer loop: i goes from 7 down to 1 */
    for (i = 7; i >= 1; i--) {
        /* Inner loop: print letters A to (A + i - 1) */
        for (j = 0; j < i; j++) {
            printf("%c ", 'A' + j);
        }
        printf("\n");
    }
    return 0;
}
```

Step-by-Step Explanation:

1. **#include <stdio.h>** — includes the standard input/output library so we can use printf().
2. **Outer loop** *for(i=7; i>=1; i--)* — starts at 7 (7 letters on first row) and decrements each time, giving us 7 rows down to 1.
3. **Inner loop** *for(j=0; j<i; j++)* — runs *i* times per row. 'A'+j converts integer offset to ASCII character (A=65, B=66, ...).
4. **printf("\n")** — moves to a new line after each row.
5. Each row prints one fewer letter than the previous row.

Q2. Multiplication Table

Problem: Write a C program to print the multiplication table of a user-entered number. Format: 10 * 1 = 10, 10 * 2 = 20, ...

```
#include <stdio.h>
```

```

int main() {
    int num, i;
    printf("Enter a number: ");
    scanf("%d", &num);

    /* Loop from 1 to 10 to print the table */
    for (i = 1; i <= 10; i++) {
        printf("%d * %d = %d\n", num, i, num * i);
    }
    return 0;
}

```

Sample Output (for num = 10):

```

10 * 1 = 10
10 * 2 = 20
10 * 3 = 30
...
10 * 10 = 100

```

Step-by-Step Explanation:

1. **scanf("%d", &num)** — reads an integer from the keyboard and stores it in *num*. The & gives the address of *num*.
2. **for(i=1; i<=10; i++)** — repeats 10 times, multiplier goes from 1 to 10.
3. Inside the loop, **num * i** computes the product and **printf** prints the formatted line.
4. **%d** is the format specifier for integers in **printf/scanf**.

Q3. Power Calculation (i power j)

Problem: Two numbers *i* and *j* are entered. Find the result of i^j (*i* raised to power *j*).

```

Enter value of i = 2
Enter value of j = 4
The result is = 16

```

```

#include <stdio.h>

int main() {
    int i, j, k;
    long long result = 1;

    printf("Enter value of i: ");
    scanf("%d", &i);
    printf("Enter value of j: ");
    scanf("%d", &j);

    /* Multiply i by itself j times */
    for (k = 1; k <= j; k++) {
        result = result * i;
    }

    printf("The result is = %lld\n", result);
    return 0;
}

```

Step-by-Step Explanation:

1. **result = 1** — initialize to 1 (neutral element of multiplication). Never initialize to 0 for multiplication.
2. The loop runs j times. Each iteration multiplies result by i .
3. After j iterations: $\text{result} = i \times i \times i \times \dots$ (j times) $= i^j$.
4. Example: $i=2, j=4 \rightarrow 1 \times 2=2 \rightarrow 2 \times 2=4 \rightarrow 4 \times 2=8 \rightarrow 8 \times 2=16$. ✓
5. **long long** is used to handle large powers without overflow.
6. **Alternative:** You can also use the math library function `pow(i, j)` with `#include <math.h>`.

Q4. Greatest of Three Numbers (Conditional Operator)

Problem: Find the greatest of three numbers entered by the user using **conditional (ternary) operators**.

```
Enter 1st value = 2
Enter 2nd value = 5
Enter 3rd value = 1
The greatest value is = 5
```

```
#include <stdio.h>

int main() {
    int a, b, c, greatest;

    printf("Enter 1st value: ");
    scanf("%d", &a);
    printf("Enter 2nd value: ");
    scanf("%d", &b);
    printf("Enter 3rd value: ");
    scanf("%d", &c);

    /* Conditional (ternary) operator: condition ? value_if_true : value_if_false */
    greatest = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);

    printf("The greatest value is = %d\n", greatest);
    return 0;
}
```

How the Conditional Operator Works:

Syntax: `condition ? expression_if_true : expression_if_false`

1. First check: **(a > b)** — Is a greater than b ?
2. If YES $\rightarrow$ check **(a > c)**: if $a > c$, $\text{greatest} = a$; else $\text{greatest} = c$.
3. If NO ($b \geq a$) $\rightarrow$ check **(b > c)**: if $b > c$, $\text{greatest} = b$; else $\text{greatest} = c$.
4. Example: $a=2, b=5, c=1 \rightarrow (2>5)? \text{NO} \rightarrow (5>1)? \text{YES} \rightarrow \text{greatest} = 5$. ✓
5. The ternary operator is a compact alternative to if-else. It **MUST** be used here as the question requires conditional operators.

Q5. Reverse a 1D Array

Problem: Five numbers are entered into array `i[5]`. Reverse and print the reversed array.

```
Input: 1 2 1 3 5
The reverse values in array = 5 3 1 2 1
```

```

#include <stdio.h>

int main() {
    int i[5], j, temp;

    /* Step 1: Read 5 numbers into the array */
    printf("Enter 5 numbers: ");
    for (j = 0; j < 5; j++) {
        scanf("%d", &i[j]);
    }

    /* Step 2: Reverse using two-pointer swap technique */
    int left = 0, right = 4;
    while (left < right) {
        temp = i[left];
        i[left] = i[right];
        i[right] = temp;
        left++;
        right--;
    }

    /* Step 3: Print reversed array */
    printf("The reverse values in array = ");
    for (j = 0; j < 5; j++) {
        printf("%d ", i[j]);
    }
    printf("\n");
    return 0;
}

```

Step-by-Step Explanation:

1. **Array Declaration:** `int i[5]` creates an array of 5 integers. Indices are 0, 1, 2, 3, 4.
2. **Input Loop:** `scanf` reads each element one by one using `&i[j]` (address of element `j`).
3. **Two-Pointer Swap:** `left` starts at index 0, `right` at index 4. They move toward each other, swapping elements.
4. **Swap logic:** `temp = i[left]` (save left), `i[left] = i[right]` (overwrite left), `i[right] = temp` (put saved into right).
5. Example trace: `[1,2,1,3,5] → swap(0,4) → [5,2,1,3,1] → swap(1,3) → [5,3,1,2,1] → left=right, stop. ✓`
6. Arrays in C are **0-indexed** — the first element is `i[0]`, last is `i[4]`.

Q6. Larger of Two Numbers Using a Function

Problem: Write a C program to print the larger of two numbers. Use a **function** for comparison and return the answer with **return()**. Pass both numbers as arguments.

```

#include <stdio.h>

/* Function declaration (prototype) */
int findLarger(int x, int y);

int main() {
    int first, second, result;

```

```

    printf("Enter first number: ");
    scanf("%d", &first);
    printf("Enter second number: ");
    scanf("%d", &second);

    /* Call the function, passing both numbers as arguments */
    result = findLarger(first, second);

    printf("The larger number is: %d\n", result);
    return 0;
}

/* Function definition – does the actual comparison */
int findLarger(int x, int y) {
    if (x > y) {
        return(x);
    } else {
        return(y);
    }
}

```

Key Concepts — Functions in C:

A function has 4 parts: Return type | Function name | Parameters | Body

1. **Function Prototype:** `int findLarger(int x, int y);` — tells the compiler the function exists before `main()` uses it.
2. **Arguments vs Parameters:** In the call `findLarger(first, second)`, `first` & `second` are *arguments*. In the function, `x` & `y` are *parameters* (copies of the arguments).
3. **return(x):** Sends the value of `x` back to where the function was called. Execution of the function stops here.
4. **int findLarger(...):** The return type `int` means the function gives back an integer value.
5. **result = findLarger(first, second):** The returned value is stored in `result`.
6. This approach uses **pass by value** — changes to `x` and `y` inside the function do NOT affect `first` and `second` in `main()`.

Quick Reference — Key Concepts

Concept	Used In	Key Syntax
Nested for loops	Q1, Q5	<code>for(i=...) { for(j=...) { } }</code>
for loop + scanf	Q2, Q3, Q5	<code>for(i=1; i<=10; i++) { printf(...) }</code>
while loop	Q5 (swap)	<code>while(left < right) { ... }</code>
Ternary operator	Q4	<code>condition ? val_true : val_false</code>
Arrays	Q5	<code>int arr[5]; arr[0]...arr[4]</code>
Functions + return	Q6	<code>int func(int x){ return(x); }</code>
Character arithmetic	Q1	<code>'A' + j</code> gives B, C, D...

Power without pow()	Q3	result = 1; for(k=1;k<=j;k++) result*=i
---------------------	----	---

Exam Tip: Always write `#include <stdio.h>` and `int main()` with `return 0`. Use proper indentation. Comment your code. The examiner awards marks for clarity.